1. **(25%)** Realice un conteo de operaciones sobre cada uno de los siguientes algoritmos, y clasifique el resultado con notación *big-Oh*, demostrando la pertenencia en cada caso. Deje evidencia de todo su procedimiento.

a.

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}
```

b.

línea 1 = 1

Ciclo de i

$i = \frac{n}{2}$ hasta n

ciclo de j

$j = 1$ hasta $\frac{n}{2}$

ciclo de k

$k = 1, 2, 4, 8 \ldots$ etc

$\log_2 n$

Resultado

$O(n^2 \log n)$

línea		conteo
1	$i = n/2$	C
2	condicion $i <= n$	$\frac{n}{2} C$
3	$i++$	$\frac{n}{2} C$
4	$j = 1$	$\frac{n}{2} C$
5	condicion de j	$\frac{n^2}{4} C$
6	$j++$	$\frac{n^2}{4} C$
7	$k = 1$	$\frac{n^2}{4} C$
8	condicion de k	$\frac{n^2}{4} \log_2 n C$
9	$k = k^2$	$\frac{n^2}{4} \log_2 n C$
10	contador ++	$\frac{n^2}{4} \log_2 n C$

```
}
```

b.

```
void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            break;
        }
    }
}
```

línea		conteo
1	n <= 1	c
2	i = 1	c
3	condicion i <= n	nc
4	i++	nc
5	j = 1	nc
6	condicion j <= n	nc
7	Printf	nc
8	break	nc

Por el break el ciclo interno se ejecuta 1 vez por iteración del ciclo de afuera

Resultado O(n)

c.

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
        }
    }
}
```

ciclo i:

$\frac{n}{3}$

ciclo j:

1, 5, 9, 13 ... n

$\frac{n}{4}$

línea		conteo
1	i = 1	C
2	condicion $i <= \frac{n}{3}$	$\frac{n}{3}C$
3	i++	$\frac{n}{3}C$
4	j = 1	$\frac{n}{3}C$
5	condicion $j <= n$	$\frac{n^2}{12}C$
6	j += 4	$\frac{n^2}{12}C$
7	printf	$\frac{n^2}{12}C$

Resultado

$O(n^2)$

3. **(25%)** Demuestre la veracidad de los siguientes enunciados:
   a. $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$
   b. $\max(f(n), g(n)) = \Theta(f(n) + g(n))$
   c. $f(n) = O(g(n)) \wedge g(n) = O(h(n)) \Rightarrow h(n) = \Omega(f(n))$
   d. $2^{n+1} = O(2^n)$

(a) Por definición

$f(n) = \Theta(g(n))$

si existe $c_1, c_2 > 0$ tal que

$c_1 g(n) \leq f(n) \leq n_2 g(n)$

$f(n) \leq c_2 g(n) \Rightarrow f(n) = O(g(n))$ y $c_1 g(n) \leq f(n) \Rightarrow f(n) = \Omega(g(n))$

entonces es Verdadero.

(b) $\max(f(n), g(n)) = \Theta(f(n) + g(n))$

Tomando

$f(n) = n \quad g(n) = -n$

$\max(f(n), g(n)) = -n$

$f(n) + g(n) = 0$

$n = -\Theta(0)$

No es verdadero a menos que

$f(n), g(n) \geq 0$

(c)

$f(n) = O(g(n))$

$g(n) = O(h(n))$

$f(n) \leq c_1 g(n)$

$g(n) \leq c_2 h(n)$

$f(n) \leq c_1 c_2 h(n)$

$f(n) = O(h(n))$ equivale a $h(n) = \Omega(f(n))$

Entonces es verdadero

(d)

$2^{n+1} = O(2^n)$

$2^{n+1} = 2(2^n)$

$c = 2$

$2^{n+1} \leq 2(2^n)$

$2^{n+1} = O(2^n)$

Entonces es verdadero